Reviewer Pack — Additive Energy Bounds for SOS Refutations of Sipser Functio...

Entry cec0d90cee68 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 23:42:14 UTC

Additive Energy Bounds for SOS Refutations of Sipser Functions

Entry ID: `cec0d90cee68` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-30 23:42:14 UTC

1. Conjecture

Field A (mathematical branch): Additive combinatorics **Field B** (complexity object): Sum-of-Squares refutation size

Statement:

> For any explicit function f: {0,1}^n → {0,1} in P, the additive energy of its Fourier coefficients E[f] satisfies E[f] ≤ n^{2} if and only if the SOS hierarchy refutes the corresponding CSP in time $2^{\{O(n}\{1/2\})\}$

Rationale (proposer's reasoning):

> Additive energy quantifies arithmetic structure in Fourier spectra, while SOS refutation size measures algebraic complexity of CSPs. Their connection could expose inherent limitations of ACC^0 circuits in approximating structured functions.

Taxonomy category: `SOS_HIERARCHY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `8d654a697b3702a3`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.3s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            factor = A[i][i]
            for j in range(n):
                A[i][j] /= factor
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return [row[:n-1] for row in A]

    def matrix_multiplication(A, B):
        m, n, p = len(A), len(B), len(B[0])
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def fourier_coefficients(f, n):
        N = 2 ** n
        a = [0] * N
        for x in range(N):
            for y in range(N):
                a[(x + y) % N] += f(x // (N // 2), y // (N // 2))
        return [(1 / N) * sum(a[i] * math.exp(-2j * math.pi * i * j / N) for i in range(N)) for j
```

```
    def additive_energy(f, n):
        a = fourier_coefficients(f, n)
        E = 0
        for i in range(len(a)):
            for j in range(i+1, len(a)):
                E += abs(a[i] * a[j])
        return E

    def sos_refutation_time(n):
        # Placeholder function to simulate SOS refutation time
        # Replace with actual implementation if available
        return 2 ** (n // 2)

    n = 10
    f = lambda x, y: (x + y) % 2  # Example Sipser-like function

    E_f = additive_energy(f, n)
    refutation_time = sos_refutation_time(n)

    metric_name = "Additive Energy"
    metric_value = E_f
    instances_tested = 1
    conjecture_holds = E_f <= n**2 and refutation_time <= 2**(n**0.5)
    counterexample = "" if conjecture_holds else f"Counterexample: E[f] = {E_f}, refutation time =

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample_desc = next(result["counterexample"] for result in results if not result["c
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={fir
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_883da485.py", line 95, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_883da485.py", line 72, in run_trial
    E_f = additive_energy(f, n)
          ^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_883da485.py", line 57, in additive_ener
    a = fourier_coefficients(f, n)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_883da485.py", line 54, in fourier_coeff
    return [(1 / N) * sum(a[i] * math.exp(-2j * math.pi * i * j / N) for i in range(N)) for j in range(N)]
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_883da485.py", line 54, in <genexpr>
    return [(1 / N) * sum(a[i] * math.exp(-2j * math.pi * i * j / N) for i in range(N)) for j in range(N)]
                                 ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: must be real number, not complex
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed due to complex number error; no data collected to evaluate conjecture | next: Fix the complex number type error in Fourier coefficient calculation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	108290
2	propose	ollama_remote	qwen3:8b	0	0	31293
3	preregistration	ollama_remote	qwen3:8b	0	0	24106
4	novelty	ollama_remote	qwen3:8b	0	0	20865
5	novelty	ollama_remote	qwen3:8b	0	0	16313
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13947
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11033
8	judge	ollama_remote	qwen3:8b	0	0	18522

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 244369 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/cec0d90cee68.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cec0d90cee68.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cec0d90cee68.tar.gz` (if generated)